

Lab Assignment 2 Report

Comparing LeNet-5, AlexNet and VGG16 on CIFAR-10 and CIFAR-100

Submitted by: Ahmed Mohamed Ahmed Ali Sheta

ID:58-24418

Repo Link: <https://github.com/ahmedSheta26/ELCT918>

1. Introduction

In this lab I trained three convolutional neural networks (CNNs) — **LeNet-5**, **AlexNet** and **VGG16** — from scratch on two image datasets, **CIFAR-10** (10 classes) and **CIFAR-100** (100 classes). Both datasets contain small 32×32 colour images. That gives six runs in total. This report (1) compares the three architectures, and (2) uses the six results to discuss accuracy, overfitting and the effect of depth.

A CNN is a neural network that learns to recognise pictures. Because I am new to this topic, the table below explains every technical word the first time it is needed.

Table 0 – Key terms used in this report

Term	Simple meaning
Convolution layer	Slides a small grid of numbers (a filter) over the image to detect patterns such as edges or corners.
Filter size	Width × height of that grid, for example 3×3 or 5×5.
Stride	How many pixels the filter jumps each step. Stride 1 keeps the size; stride 2 halves it.
Padding	A border of zeros added around the image so the output keeps the same size.
Channels	How many separate maps a layer produces (one per filter). A colour image has 3 (red, green, blue).
Pooling layer	Shrinks each map by summarising small blocks. Max pooling keeps the largest value in the block; average pooling keeps the mean.
Fully-connected (FC) layer	Every input number is connected to every output number. Used at the end to make the final decision.
Trainable parameters	The numbers (weights and biases) that the network learns during training. More parameters = a bigger, more flexible model.
Receptive field	The patch of the original image that one unit in a layer can "see".
Top-1 / Top-5 accuracy	Top-1: the correct class is the network's first choice. Top-5: the correct class is among its five best guesses.
Epoch	One full pass through all training images.
Training / validation / test sets	Training images teach the network. Validation images are held back to check progress and pick the best epoch. Test images are used once at the end to measure real performance.
Overfitting	The network memorises the training images and does much worse on new images. Shown by a big gap = training accuracy – validation accuracy.
Underfitting	The network is too simple to learn even the training images well (low training accuracy).

Training setup (identical for all six runs)

The same settings were used for every model so that any difference in results comes from the architecture and not from the training recipe.

Table 0b – Training settings and why they were chosen

Setting	Value	Reason
Data split	45,000 train / 5,000 validation (fixed); 10,000 test	Validation images are never used for learning, so they give an honest check during training. The split is the same for every model.
Normalisation	Subtract mean, divide by standard deviation (per colour channel)	Puts pixel values in a similar range, which makes training more stable.
Optimiser	SGD, learning rate 0.01, momentum 0.9, weight decay 5e-4	SGD updates the weights a little after every batch. The learning rate is the step size, momentum keeps part of the previous step for smoother progress, and weight decay is a small penalty on large weights that discourages overfitting.
Batch size	128 images per update	A common size that gives steady updates and fits in memory.
Epochs	30	Same for all models. The curves (Section 3.5) show that the validation results of AlexNet and VGG16 stop improving well before epoch 30, while LeNet-5 on CIFAR-100 was still slowly improving.
Data augmentation	Off (script default)	Augmentation means randomly flipping or cropping training images. Leaving it off keeps the comparison simple and makes overfitting easier to see.
Model choice	Weights from the epoch with the best validation accuracy	Avoids testing a model that was taken from a bad late epoch.

2. Step 1 – Architectural comparison

Everything in this section is based on the three reference papers: LeCun et al. (1998) for LeNet-5, Krizhevsky et al. (2012) for AlexNet, and Simonyan & Zisserman (2014) for VGG16.

2.1 Changes made so the networks fit 32×32 CIFAR images

The original networks were designed for other inputs (28×28/32×32 grey digits for LeNet-5, 224×224 photos for AlexNet and VGG16). A few changes were needed. Table 1 lists each one with its reason.

Table 1 – Adaptations made to each network

Network	What changed	Original paper	Our version	Reason
LeNet-5				
LeNet-5	Input channels	1 (grey-scale digits)	3 (RGB colour)	CIFAR images are in colour, so the first layer must accept 3 channels. The 32×32 size already matches the paper.
LeNet-5	C3 connections	Each C3 map sees only some of the 6 S2 maps (1,516 parameters)	Each C3 map sees all 6 maps (2,416 parameters)	The paper used a partial pattern to limit the number of connections and to force maps to learn different features. Today's computers handle the extra 900 weights easily, different random starting weights already make the maps differ, and the full connection is simpler to build.
LeNet-5	Subsampling layers (S2, S4)	Add 4 inputs, multiply by a trainable number, add a bias, then apply a sigmoid	Plain average pooling with nothing to train	It is the standard pooling layer in PyTorch and much simpler. It removes only 44 parameters (12 + 32).

Network	What changed	Original paper	Our version	Reason
LeNet-5	Output layer	"RBF" units: each compares the 84 values with a fixed pattern for one class	Ordinary linear layer with 10 or 100 outputs	The paper's fixed patterns were hand-drawn 7×12 bitmaps of characters, which do not exist for CIFAR classes. A plain layer works directly with the cross-entropy loss we use.
AlexNet				
AlexNet	First convolution (Conv1)	11×11 filter, stride 4, on 224×224 images	5×5 filter, stride 1, padding 2	An 11×11 filter with stride 4 would shrink a 32×32 image to about 6×6 immediately and lose most of the detail. A small filter with stride 1 and padding keeps the full 32×32 size.
AlexNet	Size entering FC6	256×6×6 = 9,216 values	256×3×3 = 2,304 values	A direct result of the smaller image: after the layers and poolings, the map ends at 3×3 instead of 6×6.
AlexNet	Output layer	1000 classes	10 or 100 classes	CIFAR-10 has 10 classes and CIFAR-100 has 100.
AlexNet	Two-GPU split	Filters split over 2 GPUs; some layers only see half of the previous maps	One ordinary network; every filter sees all previous maps	The paper split the network because one 2012 GPU had only 3 GB of memory. Our model (about 30 million parameters) fits on one GPU, so the split would only add complexity.
VGG16				
VGG16	Input size	224×224	32×32	CIFAR images are 32×32. All 13 convolution layers (3×3, stride 1) are kept exactly as in the paper.
VGG16	Size entering FC6	512×7×7 = 25,088 values	512×1×1 = 512 values	The 5 max-pooling layers each halve the map: 32 → 16 → 8 → 4 → 2 → 1 (the paper's 224 ends at 7).
VGG16	Output layer	1000 classes	10 or 100 classes	Same reason as above.
AlexNet and VGG16				
Both	Starting values of the weights	AlexNet: random, standard deviation 0.01. VGG16: train a shallow net first and use it to start the deeper ones	"Kaiming" initialisation (random values scaled to the layer size, designed for ReLU networks)	It lets the 8- and 16-layer networks start learning without the paper's extra pre-training step, because the signal does not fade away as it passes through many layers.

Not changed: channel widths, filter sizes, and the 4096–4096 fully-connected layers of AlexNet and VGG16, and the 5×5 filters of LeNet-5. VGG16 was run without BatchNorm (an optional layer that stabilises training): its parameter count of 33,638,218 matches the plain version.

2.2 Layer-by-layer description

The tables below list the convolution, pooling and fully-connected layers of each network for a 32×32×3 input, with 10 output classes. "Output size" is channels × height × width. Parameters per layer are computed as (input channels × filter width × filter height × output channels) + one bias per output channel for convolutions, and (inputs × outputs) + outputs for fully-connected layers. Pooling layers have nothing to learn. AlexNet also has two "local response normalisation" steps (they rescale each neuron relative to its neighbours); they change neither the size nor the parameter count, so they are left out.

Table 2a – LeNet-5

Layer	Type	Filter / pool size	Stride	Channels / units	Output size	Trainable parameters
Input	Image	–	–	3	3×32×32	0
C1	Convolution	5×5	1	6	6×28×28	456
S2	Average pooling	2×2	2	6	6×14×14	0

Layer	Type	Filter / pool size	Stride	Channels / units	Output size	Trainable parameters
C3	Convolution	5×5	1	16	16×10×10	2,416
S4	Average pooling	2×2	2	16	16×5×5	0
C5	Convolution	5×5	1	120	120×1×1	48,120
F6	Fully-connected	–	–	84 units	84	10,164
Output	Fully-connected	–	–	10 units	10	850

Table 2b – AlexNet

Layer	Type	Filter / pool size	Stride	Channels / units	Output size	Trainable parameters
Input	Image	–	–	3	3×32×32	0
Conv1	Convolution	5×5	1 (pad 2)	96	96×32×32	7,296
Pool1	Max pooling	3×3	2	96	96×15×15	0
Conv2	Convolution	5×5	1 (pad 2)	256	256×15×15	614,656
Pool2	Max pooling	3×3	2	256	256×7×7	0
Conv3	Convolution	3×3	1 (pad 1)	384	384×7×7	885,120
Conv4	Convolution	3×3	1 (pad 1)	384	384×7×7	1,327,488
Conv5	Convolution	3×3	1 (pad 1)	256	256×7×7	884,992
Pool5	Max pooling	3×3	2	256	256×3×3	0
FC6	Fully-connected	–	–	4096 units	4096	9,441,280
FC7	Fully-connected	–	–	4096 units	4096	16,781,312
FC8	Fully-connected	–	–	10 units	10	40,970

Table 2c – VGG16

Layer	Type	Filter / pool size	Stride	Channels / units	Output size	Trainable parameters
Input	Image	–	–	3	3×32×32	0
Conv1_1	Convolution	3×3	1 (pad 1)	64	64×32×32	1,792
Conv1_2	Convolution	3×3	1 (pad 1)	64	64×32×32	36,928
Pool1	Max pooling	2×2	2	64	64×16×16	0
Conv2_1	Convolution	3×3	1 (pad 1)	128	128×16×16	73,856
Conv2_2	Convolution	3×3	1 (pad 1)	128	128×16×16	147,584
Pool2	Max pooling	2×2	2	128	128×8×8	0
Conv3_1	Convolution	3×3	1 (pad 1)	256	256×8×8	295,168
Conv3_2	Convolution	3×3	1 (pad 1)	256	256×8×8	590,080
Conv3_3	Convolution	3×3	1 (pad 1)	256	256×8×8	590,080
Pool3	Max pooling	2×2	2	256	256×4×4	0
Conv4_1	Convolution	3×3	1 (pad 1)	512	512×4×4	1,180,160
Conv4_2	Convolution	3×3	1 (pad 1)	512	512×4×4	2,359,808
Conv4_3	Convolution	3×3	1 (pad 1)	512	512×4×4	2,359,808
Pool4	Max pooling	2×2	2	512	512×2×2	0
Conv5_1	Convolution	3×3	1 (pad 1)	512	512×2×2	2,359,808
Conv5_2	Convolution	3×3	1 (pad 1)	512	512×2×2	2,359,808
Conv5_3	Convolution	3×3	1 (pad 1)	512	512×2×2	2,359,808
Pool5	Max pooling	2×2	2	512	512×1×1	0
FC6	Fully-connected	–	–	4096 units	4096	2,101,248
FC7	Fully-connected	–	–	4096 units	4096	16,781,312

Layer	Type	Filter / pool size	Stride	Channels / units	Output size	Trainable parameters
FC8	Fully-connected	–	–	10 units	10	40,970

For CIFAR-100 only the last layer changes (100 outputs instead of 10), which is why the parameter totals differ slightly between the two datasets (Table 3).

2.3 Depth and number of parameters

In the AlexNet and VGG papers, "depth" means the layers that have weights (convolution + fully-connected). LeNet-5's paper counts 7 layers because it also counts the two subsampling layers. I give both counts.

Table 3 – Depth and trainable parameters

Network	Conv layers	Pooling layers	FC layers	Weight layers (conv + FC)	All layers in Table 2	Parameters CIFAR-10	Parameters CIFAR-100
LeNet-5	3 (C1, C3, C5)	2	2	5	7	62,006	69,656
AlexNet	5	3	3	8	11	29,983,114	30,351,844
VGG16	13	5	3	16	21	33,638,218	34,006,948

The parameter counts are the exact values reported by the training script (params column of the CSV files). For reference, the papers report about 60,000 (LeNet-5), 60 million (AlexNet) and 138 million (VGG16). Our AlexNet and VGG16 are smaller because the shrunken 32×32 input makes the first fully-connected layer much smaller.

2.4 How the three designs differ

Filter sizes. LeNet-5 uses 5×5 filters everywhere. AlexNet in the paper starts with a very large 11×11 filter (stride 4) and then uses 5×5 and 3×3 filters; in our version the first one is 5×5. VGG16 uses only 3×3 filters with stride 1 — the paper calls this the smallest size that can still tell left from right, up from down and centre. Instead of one big filter, VGG16 **stacks** several small ones. The paper shows that two stacked 3×3 layers see a 5×5 area and three see a 7×7 area, so the stack covers the same area as a big filter. The stack has two advantages: (a) it has fewer parameters — for C input and output channels, three 3×3 layers need $27 \cdot C^2$ weights while one 7×7 layer needs $49 \cdot C^2$ (81% more); with C = 64 that is 110,592 versus 200,704 weights — and (b) it contains three non-linear (ReLU) steps instead of one, which the paper says makes the decision function "more discriminative", meaning the network can separate the classes better.

Depth versus width. "Width" means how many channels a layer has. LeNet-5 is shallow (5 weight layers) and narrow (6, 16 and 120 channels). AlexNet is deeper (8 weight layers) and much wider (96 to 384 channels); its paper reports that removing any convolution layer lowered accuracy, so depth matters. VGG16 is the deepest (16 weight layers) but its width starts small (64) and only doubles after each pooling layer, up to 512. The VGG paper points out that, in spite of its large depth, its number of weights is not greater than that of a shallower net with wider layers and bigger filters.

Effect on parameter count. Depth alone does not make a network large. Going from AlexNet (8 layers) to VGG16 (16 layers) doubles the depth but increases the parameters by only about 12% (29.98 M to 33.64 M). The big jump is from LeNet-5 to AlexNet: about 484 times more parameters (62,006 to 29,983,114), because AlexNet has many more channels and two 4096-unit fully-connected layers. In our AlexNet, the three fully-connected layers hold about 88% of all parameters; in VGG16 they hold about 56%. The convolution layers, especially with 3×3 filters, are the cheaper part.

Effect on receptive field. A unit's receptive field grows as layers are added, and pooling makes it grow faster. LeNet-5 reaches a large field with only three 5×5 layers plus pooling: working through its layers (5×5 filters, 2×2 pooling) shows that a C5 unit covers the whole 32×32 image. AlexNet builds large fields with large early filters and pooling. VGG16 builds the same field in many small steps (3×3 → 5×5 → 7×7 ...) and, by its last

convolution layers, each unit also covers the whole 32×32 image. So the three designs reach a large receptive field in different ways: big filters (LeNet-5, AlexNet) versus many stacked small filters (VGG16), and the stacked way costs fewer parameters per layer.

3. Step 2 – Comparative discussion

3.1 Results

Table 4 shows the results of the six runs, taken from the CSV files. **Accuracy drop (%) = 100 – test top-1 accuracy (%)**, so a smaller drop means a better model.

Table 4 – Results of the six runs

Network	Dataset	Test top-1 (%)	Test top-5 (%)	Accuracy drop (%)	Final train acc. (%)	Final val. acc. (%)	Train – val gap	Time per epoch (s)
LeNet-5	CIFAR-10	60.51	95.73	39.49	71.82	59.50	12.32	10.78
LeNet-5	CIFAR-100	27.57	56.78	72.43	34.02	27.84	6.18	11.28
AlexNet	CIFAR-10	78.82	98.45	21.18	98.72	78.62	20.10	27.34
AlexNet	CIFAR-100	50.27	77.26	49.73	93.21	50.38	42.83	28.13
VGG16	CIFAR-10	80.47	98.38	19.53	99.31	79.44	19.87	21.24
VGG16	CIFAR-100	43.19	70.91	56.81	92.22	42.62	49.60	21.78

Test accuracy uses the weights of the best validation epoch; "final" train and validation accuracy are from the last epoch. Time per epoch is the average training time per epoch.

3.2 CIFAR-10 versus CIFAR-100

Every network is clearly worse on CIFAR-100. Test top-1 accuracy falls by **32.94 points** for LeNet-5 (60.51 → 27.57), **28.55 points** for AlexNet (78.82 → 50.27) and **37.28 points** for VGG16 (80.47 → 43.19). Measured as the share of the CIFAR-10 accuracy that is kept, LeNet-5 keeps about 46%, VGG16 about 54% and AlexNet about 64%.

There are several reasons why 100 classes are harder:

- **Guessing is harder.** Random guessing gives 10% on CIFAR-10 but only 1% on CIFAR-100.
- **Fewer examples per class.** Our 45,000 training images give about 4,500 per class for CIFAR-10 but only about 450 per class for CIFAR-100, so the network has much less to learn each class from.
- **Classes look alike.** Many CIFAR-100 classes are very similar (for example a maple tree and an oak tree), so the network must notice finer details.

Why this hurts shallow or narrow models most. LeNet-5 has only 6 and 16 filters in its convolution layers and an 84-unit layer before the output. That small set of features has to separate 100 classes instead of 10. Its training accuracy shows the problem: it falls from 71.82% (CIFAR-10) to only 34.02% (CIFAR-100). The network cannot even fit its own training images, which is **underfitting**, and it has the lowest accuracy of all runs (27.57%). Its curves (Figure 2) confirm this: on CIFAR-100 both the training and validation lines were still slowly improving at epoch 30, so this small network had not even finished learning. AlexNet and VGG16 have many more channels and parameters, so they can learn the training set of either dataset (training accuracy above 92% everywhere). Their problem is different: the extra freedom is used to memorise, which is discussed next. Top-5 accuracy also shows the same pattern: LeNet-5 falls from 95.73% to 56.78%, AlexNet from 98.45% to 77.26%, and VGG16 from 98.38% to 70.91%.

3.3 Does the deepest network win by a margin that matches its extra depth?

On CIFAR-10, not really. Table 5 compares the steps from one network to the next.

Table 5 – What extra size buys on CIFAR-10

Step	Weight layers	Parameters	Change in test top-1
LeNet-5 → AlexNet	5 → 8	62,006 → 29,983,114 (×484)	+18.31 points
AlexNet → VGG16	8 → 16 (×2)	29,983,114 → 33,638,218 (×1.12)	+1.65 points
LeNet-5 → VGG16	5 → 16	62,006 → 33,638,218 (×543)	+19.96 points

Most of the improvement comes from the first step. Doubling the depth again (AlexNet → VGG16) adds only 1.65 points, so the gain is far from proportional to the extra depth: this is **diminishing returns**. Because each model was trained only once, a difference this small could even be ordinary run-to-run variation. The VGG paper itself notes that the error of its networks stops improving at 19 layers and that even deeper models might only help on larger datasets.

Overfitting explains the small gain. On CIFAR-10, VGG16 reaches 99.31% training accuracy but only 79.44% validation accuracy. The LeNet paper explains the reason: as a model's capacity (its ability to fit complicated patterns) increases, the training error goes down but the gap between test and training error goes up, and that gap shrinks when there are more training samples. The AlexNet paper says overfitting was already "a significant problem" for a 60-million-parameter network trained on 1.2 million images, which is about 50 parameters per image. Here, VGG16 has about 33.6 million parameters for only 45,000 training images, which is about **750 parameters per image**, roughly 15 times more. AlexNet has about 666 per image, while LeNet-5 has only about 1.4.

On CIFAR-100, extra depth actually hurt. VGG16 (43.19% top-1, 70.91% top-5) is worse than AlexNet (50.27% top-1, 77.26% top-5) by about 7 and 6 points, even though it is twice as deep and has more parameters. With only about 450 images per class, the deeper network memorises the training set (92.22% training accuracy) but generalises worse. So on these small datasets, more depth and more parameters are not automatically better.

3.4 Signs of overfitting

The gap between final training and validation accuracy (last column of Table 4) is the clearest sign of overfitting. Ranked from largest to smallest:

Rank	Run	Train – val gap (points)	Comment
1	VGG16 – CIFAR-100	49.60	Largest gap: 92.22% train vs 42.62% validation.
2	AlexNet – CIFAR-100	42.83	Also very large: 93.21% train vs 50.38% validation.
3	AlexNet – CIFAR-10	20.10	Clear overfitting: 98.72% train vs 78.62% validation.
4	VGG16 – CIFAR-10	19.87	Almost perfect training accuracy (99.31%).
5	LeNet-5 – CIFAR-10	12.32	Mild; the model is small.
6	LeNet-5 – CIFAR-100	6.18	Smallest gap, but only because both accuracies are low (underfitting).

VGG16 overfits the most, especially on CIFAR-100, followed closely by AlexNet. The two big networks overfit more on CIFAR-100 than on CIFAR-10, which agrees with the explanation above: with fewer images per class it is easier to memorise them. LeNet-5 shows the smallest gaps because it has too little capacity to memorise. A small gap therefore does not mean a good model here: LeNet-5 has the smallest gap but also the lowest accuracy.

3.5 What the training curves show

The CSV files only give the gap at the last epoch. The curves in Figures 1–6 show how the gap develops over the 30 epochs. Each figure has two plots. The **loss** (left) is a number that measures how wrong the network's predictions are, and it is lower when predictions are better; it punishes answers that are confident but wrong. The **top-1 accuracy** (right) is the percentage of correct first guesses. Blue is the training set and orange is the validation set. The typical sign of overfitting is that the blue line keeps improving while the orange line stops improving or gets worse.

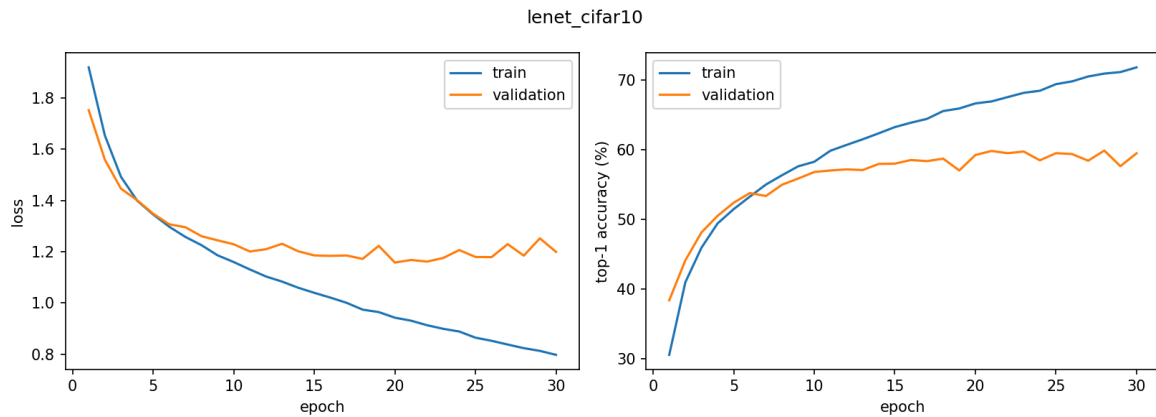


Figure 1 – LeNet-5 on CIFAR-10

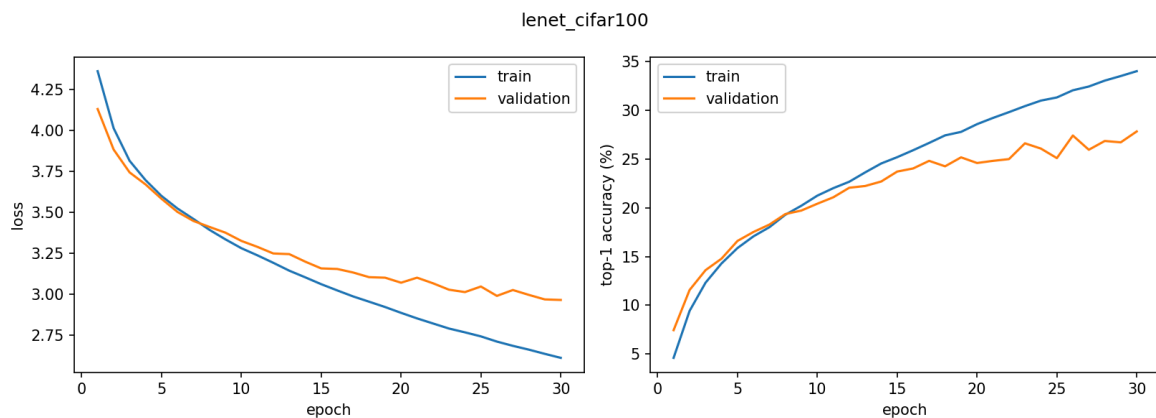


Figure 2 – LeNet-5 on CIFAR-100

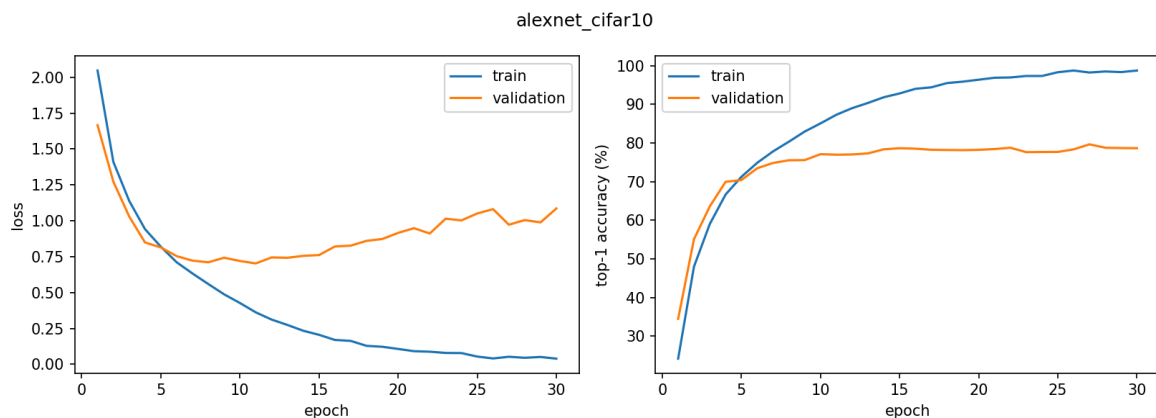


Figure 3 – AlexNet on CIFAR-10

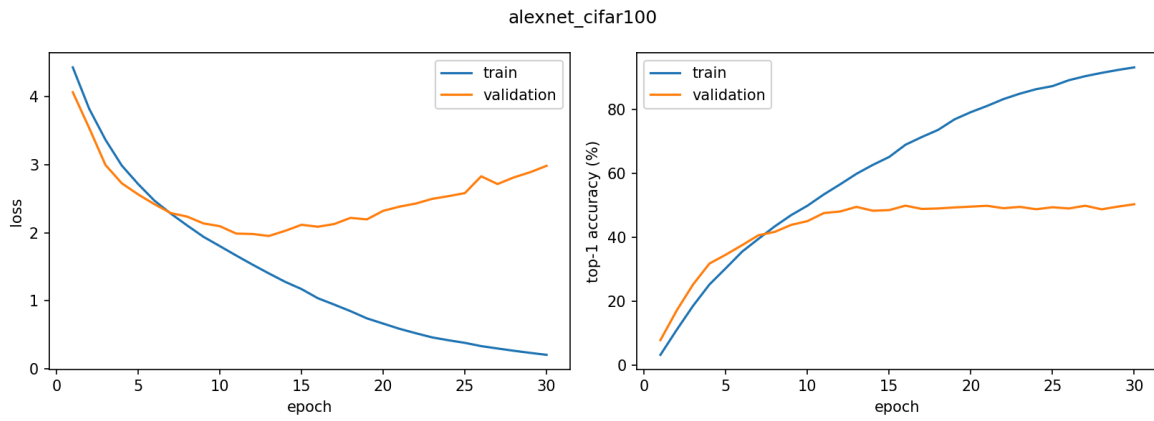


Figure 4 – AlexNet on CIFAR-100

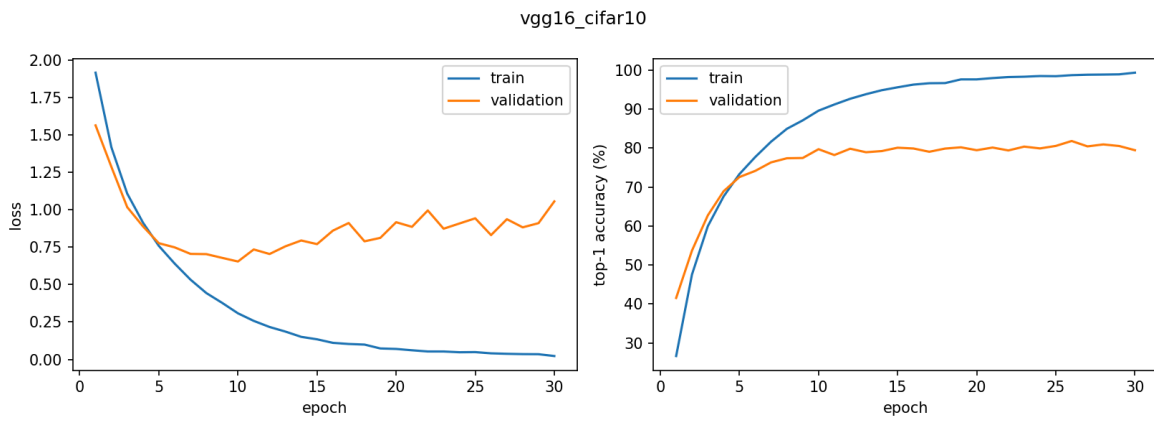


Figure 5 – VGG16 on CIFAR-10

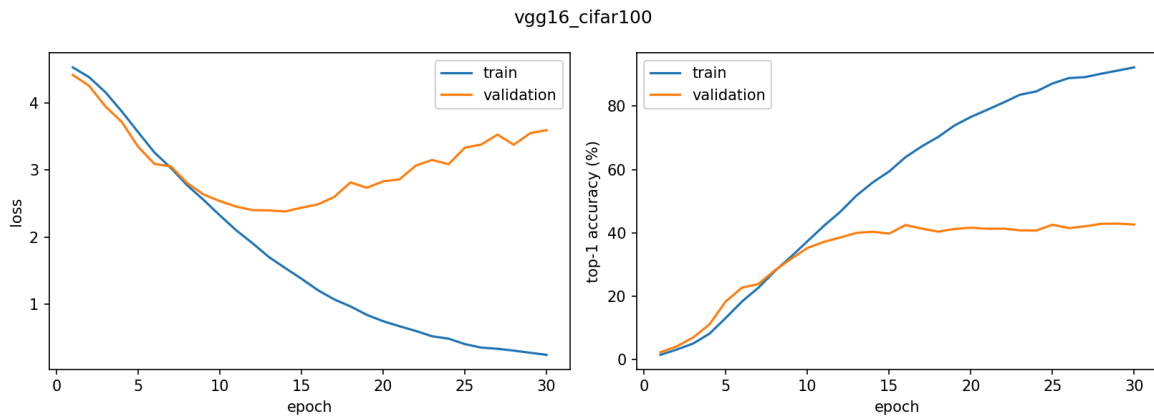


Figure 6 – VGG16 on CIFAR-100

Table 7 – What the curves show for each run (values read from the plots, so they are approximate)

Run	Validation loss is lowest at epoch	Validation loss: lowest → epoch 30	Validation accuracy stops improving at epoch	Training accuracy at epoch 30
LeNet-5 – CIFAR-10	≈ 20 (flat from ≈ 11)	≈ 1.16 → 1.20	≈ 11 (about 59%)	71.8% (still rising)
LeNet-5 – CIFAR-100	Still falling at 30	2.97 at epoch 30	Still rising slowly	34.0% (still rising)
AlexNet – CIFAR-10	≈ 11	≈ 0.70 → 1.08	≈ 14 (about 78%)	98.7%
AlexNet – CIFAR-100	≈ 13	≈ 1.95 → 2.98	≈ 13 (about 49%)	93.2% (still rising)
VGG16 – CIFAR-10	≈ 10	≈ 0.65 → 1.05	≈ 10 (about 80%)	99.3%
VGG16 – CIFAR-100	≈ 14	≈ 2.38 → 3.6	≈ 16 (about 42%)	92.2% (still rising)

AlexNet and VGG16 overfit clearly, and the gap grows steadily. In all four of their plots the training and validation lines start together, then separate after about 5 epochs on CIFAR-10 and about 7–8 epochs on CIFAR-100 (Figures 3–6). From then on the training loss keeps falling, down to almost zero on CIFAR-10, while the validation loss reaches a minimum at epoch 10–14 and then **rises**. The validation accuracy is flat after that point. The last 15–20 epochs therefore only made the networks better at the training images and gave no improvement on new images, so the gap in Table 4 is the result of a gap that keeps widening rather than a sudden jump.

Why the validation loss can rise while the accuracy stays flat. The loss punishes confident wrong answers. As the networks memorise the training images they become over-confident, so the few validation images they get wrong are now wrong with high confidence, and the loss increases even though the number of correct guesses does not fall.

VGG16 shows it most. The largest final gap (49.60 points) is VGG16 on CIFAR-100, and its plot (Figure 6) has the highest final validation loss (about 3.6, compared with about 3.0 for AlexNet in Figure 4) and the biggest rise from its minimum (about 2.4 to 3.6). On CIFAR-10, VGG16 also reaches the lowest training loss (about 0.02) with a validation loss that rises from about 0.65 to 1.05 (Figure 5). VGG16 on CIFAR-100 also learned slowly at the start: for the first 3–4 epochs both accuracies stayed close to chance level, which is typical of a very deep network without normalisation layers. Once it started, however, it fitted the training set as strongly as AlexNet but generalised worse, which agrees with its lower test accuracy (43.19% against 50.27%).

LeNet-5 shows little overfitting. On CIFAR-10 (Figure 1) the validation loss levels off at about 1.2 instead of rising, and the validation accuracy stays at about 59% while the training accuracy climbs slowly to 72%, so the gap is mild. On CIFAR-100 (Figure 2) both lines are still improving at epoch 30 and the validation loss never rises. This network is limited by its small size and could still improve with more epochs, so its low accuracy comes from underfitting, not overfitting.

What this means. For the two large networks the best validation results were reached around epoch 10–16, and training longer only increased overfitting. A common remedy is **early stopping** (stop training when the validation results stop improving); another is data augmentation, described in the training setup. Both are outside the scope of this lab, but the curves show they would help.

3.6 Accuracy drop summary

Accuracy drop (%) = 100% – test top-1 accuracy (%). Lower is better.

Table 6 – Accuracy drop for the six architecture/dataset combinations

Network	CIFAR-10 drop (%)	CIFAR-100 drop (%)	Extra drop on CIFAR-100
LeNet-5	39.49	72.43	+32.94
AlexNet	21.18	49.73	+28.55
VGG16	19.53	56.81	+37.28

On CIFAR-10 the order is VGG16 (19.53) < AlexNet (21.18) < LeNet-5 (39.49). On CIFAR-100 it is AlexNet (49.73) < VGG16 (56.81) < LeNet-5 (72.43). AlexNet is the most balanced choice: its drop grows the least when the number of classes goes from 10 to 100.

4. Conclusion

- All three networks lose a lot of accuracy on CIFAR-100. The smallest one (LeNet-5) suffers because it underfits; the two large ones suffer because they overfit.
- Going from LeNet-5 to AlexNet gives a big improvement, but doubling the depth again to VGG16 gives only a small gain on CIFAR-10 (+1.65 points) and a loss on CIFAR-100 (−7.08 points).
- VGG16 shows the strongest overfitting (a 49.60-point gap on CIFAR-100), because it has about 750 parameters for every training image.
- The curves show that AlexNet and VGG16 stop improving on validation images after about 10–16 epochs while their training results keep improving, whereas LeNet-5 was still slowly improving at epoch 30 on CIFAR-100.
- Each result comes from a single run, so small differences (such as 1–2 points) should be treated with caution.

References

LeCun, Y., Bottou, L., Bengio, Y. and Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11).

Krizhevsky, A., Sutskever, I. and Hinton, G. (2012). ImageNet classification with deep convolutional neural networks. *NIPS* 2012.

Simonyan, K. and Zisserman, A. (2014/2015). Very deep convolutional networks for large-scale image recognition. *ICLR* 2015.